

PRACTICAL-03

Aim: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

CODE IMPLEMENTATION

```
srikar@LAPTOP-KHLASUTE:~/os-lab/practical$ cat practical-03.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    printf("=== PROCESS CREATION USING fork() ===\n");

    printf("\nBefore fork():\n");
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        // Child process
        printf("\n--- CHILD PROCESS ---\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("State: Running\n");

        printf("Child State: Waiting/Sleeping\n");
        sleep(10);

        printf("Child State: Running again\n");
        sleep(5);

        printf("Child State: Terminated\n");
        exit(0);
    }
    else {
        // Parent process
        printf("\n--- PARENT PROCESS ---\n");
        printf("PID: %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("Child PID: %d\n", pid);
        printf("State: Running\n");

        printf("Parent State: Waiting for child...\n");
        wait(NULL);

        printf("Child process has terminated.\n");
        printf("Parent State: Running\n");
    }
}
```

EXECUTION OUTPUT

The code compiled successfully via GCC and demonstrated state transitions using standard system calls and timer sleeps.

```
srikar@LAPTOP-KHLASUTE:~/os-lab/practical$ gcc practical-03.c -o practical-03
srikar@LAPTOP-KHLASUTE:~/os-lab/practical$ ./practical-03

=== PROCESS CREATION USING fork() ===

Before fork():
PID: 7707
PPID: 4497

--- PARENT PROCESS ---
PID: 7707
PPID: 4497
Child PID: 7708
State: Running
Parent State: Waiting for child...

--- CHILD PROCESS ---
PID: 7708
PPID: 7707
State: Running
Child State: Waiting/Sleeping
Child State: Running again
Child State: Terminated
Child process has terminated.
Parent State: Running
Parent State: Terminated

srikar@LAPTOP-KHLASUTE:~/os-lab/practical$
```